



DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Authenticated Delay Based Geolocation of Cloud Resources

Jawad Tahir





DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Authenticated Delay Based Geolocation of Cloud Resources

Authentifizierte, verzögerungsbasierte Validierung der Geolokation von Cloud Ressourcen

Author:	Jawad Tahir
Supervisor:	Prof. Dr. Claudia Eckert, Ph.D
Advisor:	Dr. Philipp Stephanow-Gierach, Ph.D
Submission Date:	December 16, 2019



I confirm that this master's thesis in informatics is my own work and I have documented all sources and material used.

Munich,

Jawad Tahir

Acknowledgments

I would like to acknowledge a couple of people here, without their help and guidance, this thesis would not have been made possible. First, I would like to thank my supervisor Prof. Dr. Claudia Eckert for trusting my research capabilities and giving me the opportunity to work on this thesis with her. I would specially like to thank my advisor Dr. Philipp Stephanow-Gierach for his continuous guidance, support and critique on my work. I also would like to acknowledge my friend and colleague Guy Gafni for giving me his technical insight into my approach.

I take this opportunity to acknowledge my parents whose sacrifices and love for their children know no bounds. Their ambition, vision and support took me to the place, where I am today. Also, I would give acknowledgment to my brother and his wife for taking care of me, specially during the stressful days.

In the end I would like to thank all my friends who kept me sane with their laughs and talks and helped me wherever and whenever I needed.

Abstract

Cloud computing has allowed the businesses to outsource their computer infrastructure related tasks to cloud service providers without paying huge upfront infrastructure cost. Businesses have to abide by some data protection laws which may restrict the storage of the data in some geographical boundaries to protect the digital rights of the citizens. Cloud service providers, for this reason, provide with different regions to host the resources. Different regions can have different operating cost for a cloud service provider. A malicious cloud service provider, motivated by lower operating cost, can host/relocate the resources to a lower operating cost region which does not reside in the restricted geographic boundaries. In this thesis we propose an authenticated delay based geolocation validation tool which utilizes state-of-the-art machine learning techniques to validate and even predict the location of a cloud resource.

Contents

Acknowledgments	iii
Abstract	iv
List of Figures	vii
List of Tables	viii
1 Introduction	1
2 Litratue Survey	4
2.1 Conclusion	7
3 Background	8
3.1 Case for Validation	8
3.2 Case for Authenticated Delays	10
3.3 Attack Model	13
3.4 Problem Statement	13
3.5 Proposed Approach	14
4 Experimentation Setup	17
4.1 Tools and Technologies	17
4.1.1 Paramiko	17
4.1.2 TensorFlow	18
4.1.3 Docker	18
4.2 Setup	19
4.2.1 Network Setup	19

Contents

4.2.2 Data Collection	20
4.2.3 Validation Data	21
5 Results and Evaluations	23
6 Conclusion	41
6.1 Shortcomings	41
6.2 Future Recommendations	42
Bibliography	43

List of Figures

3.1 SSH Action sequences. (courtesy wikimedia commons)	12
3.2 AWS global infrastructure [28]	14
4.1 Landmarks locations	20
5.1 Classifier accuracy over periods	24
5.2 Frequency graph of authenticated delay	25
5.3 Frequency graph of authenticated delay	26
5.4 Post-filtering frequency graph of authenticated delay	27
5.5 Post-filtering frequency graph of authenticated delay	28
5.6 Post-filter training accuracy over periods	29
5.7 Average delay over a week	30
5.8 Average delay over a week	31
5.9 Final training accuracy over periods	37
5.10 Confusion matrix	38
5.11 Confusion matrix of relayed dataset	39

List of Tables

5.1	Feature vectors	23
5.2	Feature vectors	32
5.3	Delay measurement example	32
5.4	Feature vectors	33
5.5	Feature vectors	34
5.6	Feature vectors	35
5.7	Feature vectors	36

1 Introduction

This chapter serves as the summary of the whole thesis. I would briefly explain the background of the problem. Then a few state-of-the-art and related papers tackling the problems are summarized. After that we will informally construct the problem statement and proposed approach.

Cloud computing has enabled businesses worldwide to delegate their infrastructure related requirements to a cloud service provider like Amazon [1] , Microsoft [2], Google [3] to name a few. Now businesses can liberate themselves from tedious, expensive and laborious tasks of infrastructure by using cloud services like storage and compute of these service providers. Businesses generally have to comply with legal requirements regarding electronic data usage and storage. For example European Union (EU) Data Protection Directive prohibits the flow of personal data out of EU member states to any geographic region which does not have an adequate level of protection of data [4]. On the same note, Australian Privacy Principles (APP) forbids the flow of the data to foreign country whose laws are not substantially similar to APP [5]. Keeping these requirements in mind, cloud service providers allow customers choose different zones hosted in different geographical regions in the Service-Level Agreement (SLA). However customers have to rely on service providers guarantee as these service provider does not provide any verification platform for data geolocation.

Geolocation estimation and validation of cloud services components is an active research area. Gueye et al [6] proposed a Constrained-Based geolocation model. They measured the time taken of a packet from a host to multiple geographically known landmarks and then used these readings to estimate the geolocation of the host by constraining the space using Best line estimation. Stephanow et al [7] purposed a Continuous Location

Validation platform using machine learning techniques. They measured the time taken from a landmark to different regions and train the initial classifier. This classifier is then used to predict the location of different samples. They also updated the classifier with the new readings so that it also catered the dynamic changes in the cloud network.

Different [8][9][10][11] (discussed and explained in detail in chapter 2) delay-based location estimation and validation techniques rely on measurements of time taken of a packet from landmark to a cloud hosted in a zone. However they used ICMP ECHO messages to measure the time, which in cloud infrastructure usually goes till the gateway machine. A service provider may have different resources at different geographic locations, all of them accessible through a gateway at a different location. This topology results in misleading delay-based measurements which in turn distorts training of classification in case of machine learning based techniques and incorrect calculation of best line in case of constrained based techniques. This oversight may allow malicious service provider to only pretend to deploy resources at a particular location while, in fact, it is not. In more practical terms, a malicious service provider can claim to have a compute resource in Germany to capture European market but have only a gateway machine. Inclined by financial gains, it can choose to host its compute farm in a cheap labour country whose laws may not offer the same data protections as EUDPD.

In this thesis, we propose a measurement model which is based on authenticated resource. ICMP ECHO messages are not a reliable source of delay measurements as the authenticity of the resource is not guaranteed. To have some guarantees that the resources targeted during measurement collection is indeed the intended one, measurement platforms have to support authentication of the resources as well. An SSH connection is a direct authenticated connection between two peers using Public Key Infrastructure (PKI) where as a simple ICMP ECHO request, on the contrary, is usually time measurement between landmark and gateway computer. An SSH connection is to be established between host and landmark to obtain authenticated delays. To get the authenticated delay we measure the time difference between sending SSH_MSG_KEXDH_INIT and exchanging Diffie-Hellman keys between two peers as defined in RFC 4253 [12]. For this, we modified an open source implementation [13] of SSH protocol in Python to create the measurement dataset. Network delays are

variable and heavily influenced by network congestion. Network congestion can also be depend upon time of the measurement as network may more than usual traffic during peak hours, so we decided to use time as well as delay measurement in our dataset to model our network. This dataset was then used to train a neural network classifier. This classifier was then used to predict and validate the locations of different cloud service resources.

In the subsequent chapters, this methodology is discussed in detail. In chapter 2, we will review and critique the current geolocation proposed techniques. Chapter 3 builds the background and discusses our proposed approach in detail. Chapter 4 explains the implementation and experimentation set up. Results are then analysed in chapter 5. Conclusion and future work is explained in chapter 6.

2 Literature Survey

Geolocation of internet hosts, in general, and cloud services specially is an active research area. Several methodologies and tools, from DNS lookups to constraint based multilateration model, were proposed. In this chapter we will first summarize and then critique a few state-of-the-art and classical techniques for geolocation.

Geolocation of internet host is one of the oldest research topics in computer networks. One of the first approach to geolocate internet hosts was the introduction of DNS LOC [14] extension for DNS record in RFC 1986. However DNS LOC failed to reliably geolocate internet hosts as it failed to pick the pace and it was not utilized by the internet community. Next intuitive idea to geolocate internet hosts is exhaustive tabulation of IP addresses, domain names, and Autonomous System (AS) numbers to geographic locations as proposed by Moore et al [15] . However this idea is difficult to scale due to the sheer scale of internet. Also, this table would also be cumbersome to keep it updated because of the dynamic nature of the internet, hence leaving the table in an inconsistent state and making it useless to geolocate.

Padmanabhan et al [10] purposed a IP2Geo suite for geolocating Internet hosts which comprised of three individual techniques namely, GeoTrack, GeoPing and GeoCluster. GeoTrack used traceroute [16] to get the list of the routers between source and target. Location of these routers were resolved using hints found in router name. As not every router have location hints, so target is associated to the location of the nearest known router. The accuracy of this tool depends upon the distance of last known router from target. GeoPing assumes that host geographically collocated have similar delay profile. They measure the ping time to target from geographically known landmarks. GeoPing estimates the location of target to be same as of the node which has the same ping delay

value. GeoCluster is a lookup technique which maps groups of IP to geographical proximity. They gather this dataset from user registration to a popular email service. They assign the IPs of users to their home state hence assigning clusters of IPs to a geographic location. This lookup technique suffers from general problems with IP lookup tables like scalability and maintainability. Also a user may provide wrong/stale location while registering.

In a highly cited paper by Gueye et al [6], they proposed a Constraint Based Geolocation (CBG) methodology to give probabilistic fencing of geolocation of internet hosts, which is majorly inspired by RADAR [6][17]. Like RADAR, they used multilateration technique to locate resources. In their approach they have multiple geographically known landmarks. They have the baseline of distance-delay graph as two-thirds of the speed of light which is the highest speed on which information can travel in fiber optic cable. But because of multiple unavoidable factors, information almost never travels at that speed, hence they measure best line from the readings which gives lower bound of delays. They used dataset created by Bovy et al [18] which used IP layer traceroute to measure delay between a geographically known landmark and target location. For geolocation they measure the delay from a set of landmarks to target. Estimation of location is done through best line from each landmark. Physical area is then constrained using multiple landmarks.

In the similar footsteps, Katz-Bassett et al [9] proposed Topology Based Geolocation (TBG) which also measure topology of the network as well as delays to geolocate. In TBG, maximum possible area is defined by maximum transmission speed of packet in fiber, which provides an upper bound of possible geographic region. This region is further refined using inter-router latencies along the path from the target to the landmark, obtained from the traceroute command. The final target location is obtained through a global optimization that minimizes average position error for the target and the routers.

One more noteworthy geolocation of internet host technique, before we move to geolocation of cloud services, is by Arif et al [19]. They further extended the concept of CBG in their proposed approach GeoWeight. After close inspection of ICMP delay measurements taken on PlanetLab test bed, they observed that some paths in information routing is more probable than the rest of the paths. They used this information to add further constrain to original CBG and showed that their GeoWeight outperforms CBG.

In the field of geolocation of cloud resources, Eskandari et al [8] proposed VLOC, a regression model based validator trained on delay measurements from multiple landmarks. The tool consisted of three modules. Data collector module was used to get geographic information from an IP geolocation module (The used IP address API as geo location service [20]). Measurement module measured the time a HTTP packet takes from target virtual machine to other landmarks. A distance estimate function was used to get the distance between landmark and target based on RTT measured by measurement module. At least three readings were used to triangulate the location of target using distance estimate function whose constants are derived by a regression model. This VLOC is installed on a virtual machine on cloud and trained per given landmarks. The derived distance estimate function is then used to validate its location using triangulation. However, we will see in later chapters, delay measurement is not a linear function of distance. It also depends on the structure of the network as well. Some branches of internet are faster than others and a delay measurement may mean different distances in different branches of internet. Hence a model trained on a geographic location may not work correctly in other locations. Also, HTTP readings are not reliable as they don't provide any authentication of the machine.

Both of these problems were addressed by Gondree et al [11]. The proposed a Provable Data Possession (PDP) based delay measurement which is the time it takes to verify that our communication machine is the intended machine. Latency-distance model was built on each landmark making the tool network topology aware. The data owner tag the data and store it on target service. The dataowner randomly chooses some challenges and distribute it among landmarks. Landmarks then measure the time it takes to reply with the proof. If all the proofs were accepted then each landmark provides constraint area based on it latency-distance model. The constraints were resolved to get the probabilistic location of the data.

In case of cloud services, sometimes exact location of internet hosts is not required, rather a validation of current location is enough. Staphanow et al [21] proposed a continuous location validation which keeps updating Adaptive Location Classifier. Classifier predicts the location of the resource based on delay profile. In the first step delay measurements are collected. For delay measurements they used delay in transmission layer and internet

layer from host to landmarks. They utilized ICMP ECHO and TCP SYN packets to get the delay measurements. These measurements are used to train a classifier which is then continuously updated to predict up-to-date location of a cloud resource. As each output point is a function of its delay profile, this classifier may have a hard time classifying two different locations having the same delay profile (an equilateral / isosceles triangle network configuration, for example).

2.1 Conclusion

In this chapter, We have summarized multiple classical and state-of-the-art delay-based geolocation techniques of internet hosts and cloud resources. We studied multiple techniques, from exhaustive tabulation to constraint based geolocation, for internet host geolocation. We then further summarized a few techniques to geolocate cloud resources where we have different constraints and relaxation. We saw that why PDP is required in cloud network configuration. In the end we glanced over location validation of cloud resources rather than location estimation. These resources provided helpful insights to the problem solution and proved helpful during the research and development of the current paper.

3 Background

In this chapter, we will first build a case for validation as opposed to location estimation done by numerous authors. We will formally formulate the problem before proposing a solution. A few related concepts to solution are also briefly explained.

3.1 Case for Validation

As we have seen in the literature survey, most of the work done in geolocation is about location estimation of a resource through different techniques. Location estimation gives the probabilistic region in which resource could reside. Gondree et al [11] were able to estimate the location of a resource with 90% accuracy in 626km area. This result is way too big to geolocate a single internet host, however it can be used to estimate the location of cloud resources as cloud resources are stored in huge data centers with known state / city level location info [22]. However as the location of each cloud resource is a priori known / specified at creation time, we can tackle this problem from another angle. Instead of estimating the location of a cloud resource to known location, we can validate if the resource is at known location or not. An invalidated location validation means that the service provider is serving the resource from a different location than it claims to be.

Classification in machine learning is a supervised learning technique. In supervised learning technique, we initially have some labeled data points. We then train our model on these labeled data points. The model learns the patterns from the labeled dataset and builds a function through which it can assign a label or class to unlabelled data points based on learned patterns. One very basic example of classification could be classifying emails to spam or not-spam classes by some email service provider. An email service provider

will give known spam emails (more technically, features extracted from these emails like keywords used, sender, number of recipients etc) as an input data set to classification model. The model will learn the weights for its classification function to classify emails to spam or not-spam classes from these given emails. The email service provider then predict the class of future emails with this learned model. Classification has an inherent constraint that it requires large amount of labelled data which is sometimes quite expensive and labouring task. Email service providers get this labelled data from the customers in the form of a button as “mark as spam”. Whenever the customer presses this button, it is providing the service provider with labelled data. The more the labelled data is provided by the customers, the better is the classification.

We can model our geolocation validation problem to a classification problem. We can treat each location as a separate class and we can train a model which predicts the location of resource based on its delay profile. As explained above, we require a labelled dataset for classification. For that, we can build a data collection service which will get the delay measurements and tag them with appropriate labels. This data collection service is explained in detail in experimentation chapter.

Deep learning is a subset of statistical machine learning. It is an architecture of machine learning which is modeled after human brains and consists of artificial neural network to recognize patterns in the data. A neural network is an interconnection of artificial neurons, generally referred to as nodes, which mimics the behaviour of biological neural networks. These neurons works like a biological neuron as it takes an input signal and combine it with its internal state using activation function (a nonlinear function) and produce an output using an output function. The connections between these different nodes works as a data bus and transfer the output of a node to input of next node. These connections have weights associated with them which helps to amplify or suppress the signals. These nodes are generally organized in different layers and each layer learns different features of the data and previous layer pass its feature extraction to the next layer. Given enough data and computing power, Deep Learning is preferred over simple machine learning. The reason for deep learning preference over simple machine learning techniques is that deep learning models train themselves over features present in the data rather than manually defined

task-specific algorithms. Deep learning classifier models can learn the features in the dataset that may require explicit manual extraction of features in simple machine learning classifier. We decided to use a neural network based classifier for the aforementioned reasons.

3.2 Case for Authenticated Delays

A malicious cloud service provider, driven by financial gains or lower labor cost, may host a customer's resource (from location L_a) at Location L_b (with much lower data protection rules as compared to L_a) while claiming it to be deployed on Location L_a . As data is now hosted at location L_b , it is subjected to laws and governance of location L_b . This may violate the data protection rights of customer at location L_a . Cloud service providers do not offer any Proof of Origin (PoO) of resources retrieved. Most of the delay measurements based location estimator discussed in the previous chapter can be countered by utilizing a Gateway at location L_a because they used non authenticated messages to measure time. Response message may not be generated by the target machine of request message, instead could have been replied by the gateway machine. We need some sort of Proof of Authentication from target machine to ensure the validity of the response.

We need to build a trustworthy connection from host to target before getting delay measurements. For this we need to introduce the concept of authenticated delays which allow us to verify that measurement messages are originating and replying from correct hosts. Authenticated delays can be obtained by measuring delay of packets used within application layer protocols which already provide the security property authenticity. For this paper we used SSH for authentication and delay measurement is obtained by observing time differences between different phases of SSH connection initiation.

SSH is a cryptographic network protocol for operating network services securely over an unsecured network [23]. Typical applications of SSH includes remote command execution, login and secure file transfer. Complete SSH protocol is defined in RFC 4251 to RFC 4255 [24][23][25][26][27]. SSH provides a secure channel over an unsecured network in a client-server architecture, connecting an SSH client application with an SSH server

[25]. SSH protocol consists of three layers, connection layer, authentication layer and transportation layer. SSH-CONN sits between SSH server and SSH-AUTH module and is responsible for multiplexing different encrypted logical channel over authenticated connections. This layer manages the SSH session, session multiplexing, X11 forwarding, TCP forwarding, shell, remote program execution, invoking SFTP subsystem. SSH-AUTH module is responsible for SSH client authentication at server. It uses an established connection and sits on top of SSH-TRANS module. It provides several mechanisms for user authentication. These include password authentication, public-key or host-based authentication mechanisms, challenge-response, pluggable authentication modules (PAM), Generic Security Services API (GSSAPI) and even dongles. SSH-TRANS works over TCP connection and is responsible for server authentication, confidentiality and data integrity. It does this through algorithm negotiation and a key exchange. The key exchange includes server authentication and results in a cryptographically secure connection. A typical interaction between different actors of SSH are shown in the figure 3.1

3 Background

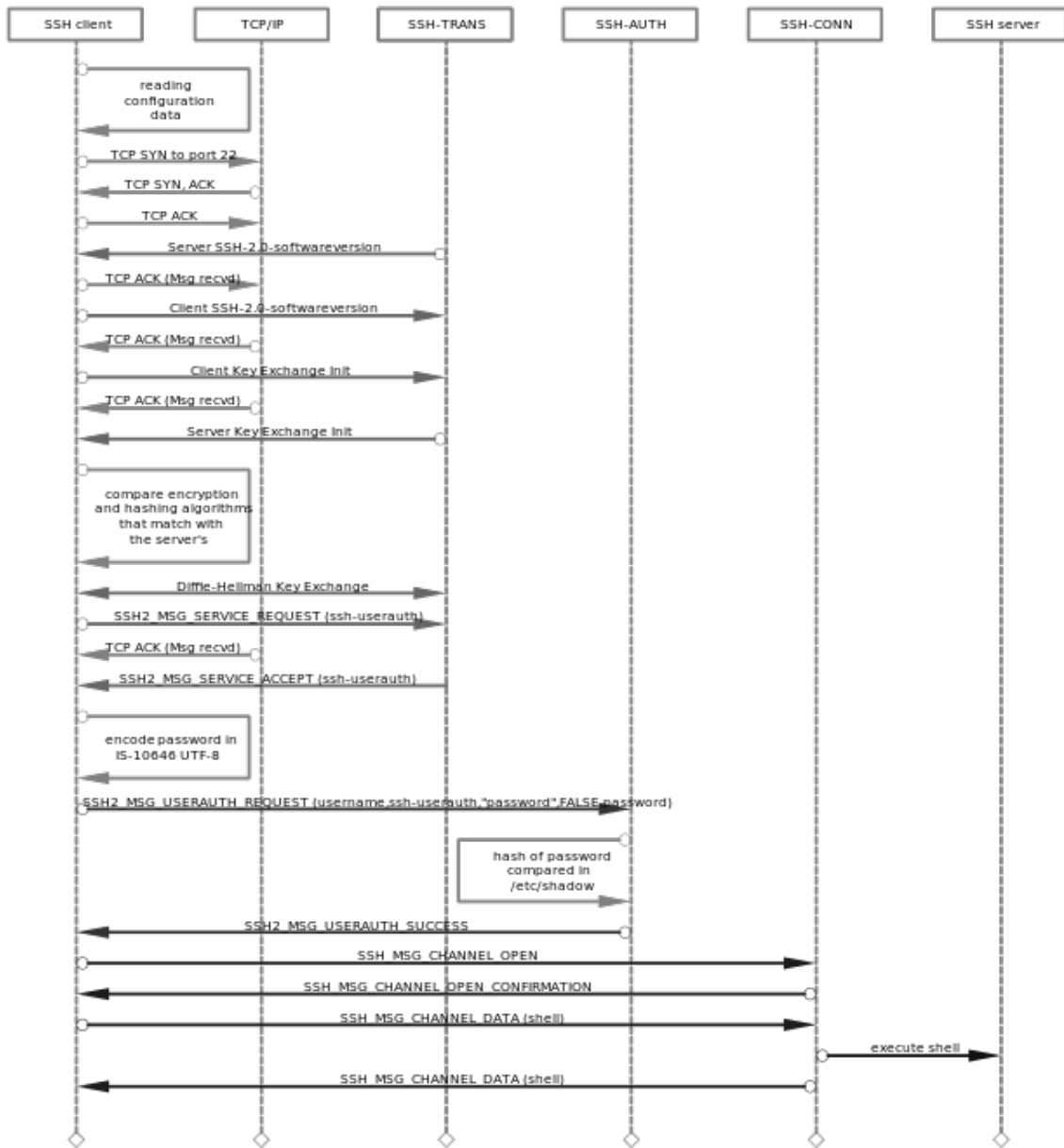


Figure 3.1: SSH Action sequences. (courtesy wikimedia commons)

In a typical interaction, client and server initiate an SSH connection which decides the SSH protocol version. Next supported key exchange algorithm are proposed and decided. Diffie-Hellman key exchange is used to verify server and generate session keys. Subsequent messages are encrypted with session keys. User is then verified through SSH-AUTH and

in case of success, an encrypted channel is opened on server. As the authenticity of the server is determined in the Diffie-Hellman key exchange, we will attribute the time it takes to exchange keys between client and server as authenticated delays. In this step the client initializes a Diffie-Hellman Key Exchange. The server then computes a shared secret which is concatenated with his public host key and other information and hashed. The server signs this hash using his private host key and sends the signature, its DH parameters as well as its public host key to the client. The client can now also compute the shared secret and verify the server's host key, thus providing server authentication.

3.3 Attack Model

A malicious cloud service provider can deceive their customers in two ways. They can either falsely advertise the location of their data centers or they can place a gateway machine. In the first scenario a cloud service provider can claim to host the cloud resources at DC_a where as they are actually hosting the cloud resource at DC_b which doesn't have the same data privacy law. In the second scenario a cloud service provider can actively try to cheat their customers and try to hide the evidence by placing a gateway machine at customers desired data center DC_a which acts as a router and reroutes the traffic to another DC_b . So the customer gets the wrong notion of location of the cloud resource.

3.4 Problem Statement

A cloud service provider have n data center $DC_1, \dots, DC_n$ in n different locations $L_1, \dots, L_n$. These n locations have m ($m \leq n$) different data protection guidelines DP_1, DP_m . A customer of cloud service has to ensure DP_j (where $1 \leq j \leq m$) as a legal requirement for his business. For this, he can only host his data on one of location L_h out of all the locations $L = L_a, \dots, L_b$ (where $1 \leq a \leq b \leq n$) which conforms to DP_j . A higher data protection guideline may cause higher production cost for a service provider. Driven by financial gains or lower labor cost, a malicious service provider may host resources at another location L_f which does not conform to DP_j while claiming to host it on L_h . This may cause violation of

data rights of citizens in DP_j .

As cloud services does not provide any Proof of Origin with resource, there is an inherent need to geolocate resources on cloud. As we explained in the previous sections, validation of the current location can be an alternative to location estimation in cloud computing. After explaining all the terms before, we can model our problem statement as follows:

Given that a customer hosts some resources at Location L_h , he must be able to reliably validate location of his resource.

3.5 Proposed Approach

Cloud service infrastructure is generally known to the customer a priori. One example of AWS can be seen in the figure 3.2

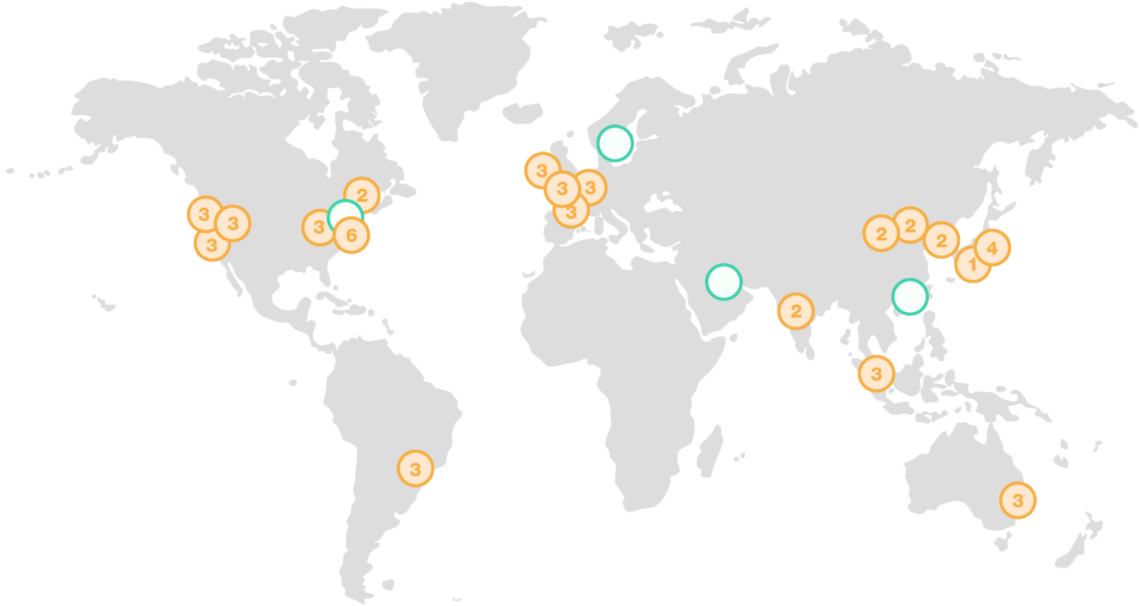


Figure 3.2: AWS global infrastructure [28]

To enable customers to validate their location, we proposed a neural network based multi class classification model which is trained on authenticated delays. This system is comprised of three modules; data collection, learning module and validation module. We will briefly explain these modules here.

Data collection: As explained in the previous chapter, all delay based geolocation rely on some sort of geographically known landmarks and delay is measured between query location and landmarks. As we a priori know the locations of all possible locations to host the resources, we set a landmark at each of the possible location. We can employ different techniques for this like we can either use independently deployed machines near their locations to use as landmarks or we can create a virtual machine at each location. At each specific time interval, each landmark measures authenticated delay to every other landmark in the network. So we can get the complete network delay snapshot at each instance. We utilize SSH protocol to measure the authenticated delays. As we are utilizing authenticated delays, the authenticity of the resource is established first. Hence even if a cloud service provider is rerouting the messages from an intermediate location, authenticated delays will measure the time it takes to get to the actual resource as only the actual resource holds the valid key to establish a connection.

Learning: These measurements are then used to train a classifier. Distance is not a linear function of delay, as multiple factors can influence the delay like topology of the network (a node connected to two different nodes may have different bandwidth at each node) and the time of the day (delay might be higher in peak office time than off-peak time) and day of the week (work days may have higher delay than weekends due to congestion). Hence a linear classification model may not be enough to learn all these non-linearity. Hence we decided to employ multi-class deep neural networks classifier to validate the location of the query machine. Here each location represents one class and number of classes represent total locations in the network. We can use DNN classifier to model the whole network.

Validation: The last module of our proposed system is validation. A customer can use this module to validate the location of his resources. He can select the expected location of his resource, validation module then exclude the landmark at that location and instructs the remaining landmarks to build a delay snapshot of the network. Classifier trained in previous step is then used to predict the location of the resource based on delay snapshot of the network. If the expected location does not match with the predicted location, it implies that cloud service provider has moved the resource to a different location without

informing the customer.

In the next chapter we will explain the implementation details of the system and experimentation set up. We will also discuss the results of our proposed approach.

4 Experimentation Setup

In this chapter we will first talk about the tools and technologies used in our proposed approach. After that we will explain the experimentation set up. Last part of the chapter is dedicated to discuss the results.

4.1 Tools and Technologies

4.1.1 Paramiko

Paramiko is a Python implementation of the SSHv2 protocol, providing both client and server functionality. While it leverages a Python C extension for low level cryptography, Paramiko itself is a pure Python interface around SSH networking concepts [29]. However we were not interested in creating only an SSH connection but also measure the time taken between different phases of keys calculation, validation and exchange in the transport layer of the SSH. Paramiko has an open source implementation hosted at Github [30]. We forked the source code from their repository and modified the source code of Paramiko and inserted some time measuring probes in the transport layer. These probes were used to measure the time taken between different phases of key exchange. We used time library of Python to measure the time. The open source implementation of modified version of Paramiko is hosted on the Github on one of the authors profile [13]

We inserted a total of 6 time measuring probes (T0, T1, T2, T3, T4, T5). T0 initializes the clock and taken right after TCP handshake and right before client HELLO message. T1 and T2 are taken after sending client HELLO and server HELLO message respectively. T3 denotes the KEXINIT (key exchange initiation) event. T4 and T5 measures the time it takes to send the KEXDH_INIT (initialization of key exchange through Diffie-Hellman client) and

receive KEXDH_REPLY (response of key exchange initialization which also includes keys of server). We denote this reading as time vector. SSH connection is established after this step and client and server can transfer information which is encrypted with these keys.

4.1.2 TensorFlow

TensorFlow is an interface for expressing machine learning algorithms, and an implementation for executing such algorithms. A computation expressed using TensorFlow can be executed with little or no change on a wide variety of heterogeneous systems, ranging from mobile devices such as phones and tablets up to large-scale distributed systems of hundreds of machines and thousands of computational devices such as GPU cards. The system is flexible and can be used to express a wide variety of algorithms, including training and inference algorithms for deep neural network models, and it has been used for conducting research and for deploying machine learning systems into production across more than a dozen areas of computer science and other fields, including speech recognition, computer vision, robotics, information retrieval, natural language processing, geographic information extraction, and computational drug discovery [31].

TensorFlow implements Keras API specification. Keras is a high-level neural networks API, written in Python and capable of running on top of TensorFlow [32]. TensorFlow also provides pre-made estimators. We used DNN classifier provided by TensorFlow to train a neural network based classifier to classify locations.

4.1.3 Docker

Docker is software containerization service. A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another. A Docker container image is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings [33]. Docker exploits cnames and namespaces provided by Linux kernel to sandbox runtime environments. As we were to work with multiple machines and we had to deploy our data collections service

to multiple endpoints, manually deploying the code to each machine was a cumbersome and time consuming task. To streamline our deployment of code to different machines, we decided to use Docker containers.

4.2 Setup

4.2.1 Network Setup

We used Microsoft Azure as our cloud service provider. The idea is to have a machine at every offered region so that a customer can have a complete view of service provider infrastructure and any change in infrastructure is detected. However to build a small prototype for this thesis we modeled our service provider to have 6 regions. We hosted machines at each of these locations. We used the following locations of Microsoft Azure to host our data collection machines:

- Johannesburg (ID: AFNORTHSAFH00, Label: 0)
- Victoria (ID: APEASTAUSFH01, Label: 1)
- Singapore (ID: APSEASTASFH02, Label: 2)
- Pune (ID: APCENTINFH03, Label: 3)
- Ireland (ID: USEASTUS1FH04, Label: 4)
- Virginia (ID: EUWESTEUFH05, Label: 5)

The network infrastructure can be visualized in the figure below.

4 Experimentation Setup



Figure 4.1: Landmarks locations

4.2.2 Data Collection

We utilized each of the above mentioned location for data collection. An SSH connection was established from each of these locations (We call these locations as our landmark) to all other landmark at any given time. We used interval of 60 seconds between each of these measurement i.e an SSH connection was established between a given landmark to all other landmark after every minute. Hence at each interval, 30 data points were collected. Each data point consisted of name of originating landmark (node_name), name of the target landmark (dest_name), a time vector (m_val), the timestamp of the time vector (m_time), time stamp of the request (r_time) and type of time vector (m_type). One example of a data point can be seen below:

```
1 {  
2     "_id" :ObjectId("5d6cb17064804543328180be"),  
3     "node_name" : "EUWESTEU05",  
4     "dest_name" : "USEASTUS1FH04",  
5     "r_time" : "2019-09-02 06:06:40.148488",
```

```
6     "m_time" : "2019-09-02 06:06:40.156579",
7     "m_val" : {
8         "t0" : 0.58587,
9         "t1" : 0.585953,
10        "t2" : 0.588141,
11        "t3" : 0.589673,
12        "t4" : 0.642019,
13        "t5" : 0.645226
14    },
15     "m_type" : "SSH_DELAY"
16 }
```

This data point shows that an SSH connection was established from a landmark name EUWESTEU05 to USEASTUS1FH04 on September 2, 2019 at 06:06:40 in the morning with `m_val` as its time vector. As all the landmarks were in different time zones and we were trying to capture a snapshot of network delays at a specific time instance, a global consensus of time resolution was required. We decided to use UTC timezone for timestamping rather than local timezone so that timestamp from Virginia to Victoria is same as Victoria to Virginia. We ran the data collection service for a month and we collected a total of 1410190 data points.

4.2.3 Validation Data

As explained in the previous chapter, a malicious service provider can deceive their customers in two ways. They can either advertise false location of the regions or they can put a gateway machine at advertised location and route the traffic to actual machine hosted to different regions. Validation data for the first scenario is quite is easy to gather. We can simply give false landmark to `dest_name`. This would represent that a cloud resource is hosted to different location but cloud service provider is claiming to host it in different locations. To mimic the second scenario we decided to use SSH port forwarding. We selected USEASTUS1FH04 to route the SSH connection to AFNORTHSFAFH00. We

forwarded one port of USEASTUS1FH04 to AFNORTHSAFH00 and when customer tries to send a SSH connection request to USEASTUS1FH04, it forwards the connection request AFNORTHSAFH00 so that client is unaware that his connection is routed to a different location. We ran the data collection service again for a day to collect the data points for the second attack model.

5 Results and Evaluations

As a network can have different delays on different path of it, we first decided to build a delay model at each of our landmark. Each model will learn how much time it takes to establish an authenticated connection to a specific landmark from all other landmarks. We trained each model on the following feature vector:

Table 5.1: Feature vectors

td	label
T_{al}	A
T_{bl}	B
T_{cl}	C
T_{dl}	D
$\vdots$	$\vdots$

Here T_{al} describes the time it takes to establish an SSH connection from landmark A to landmark L and label tells the originating landmark. Here our input feature, x , is authenticated delay and our label y , is landmark which initiated the connection. Hence the model learns that, at a given landmark, if it takes x amount of time to establish an SSH connection to current landmark then it was originated from landmark y . Our plan was to send an SSH connection request to all landmarks from our query location (the location we are trying to validate) and measure the time and then based on measured delay predict the originating location at each landmark. If the prediction turns out to be query location at every landmark model then we can say that infrastructure hasn't changed and the location of cloud resource is valid. We gave above mentioned feature vector to our classifier and we

got 78% accuracy. The graph of accuracy over training period at one landmark can be seen figure 5.1 below.

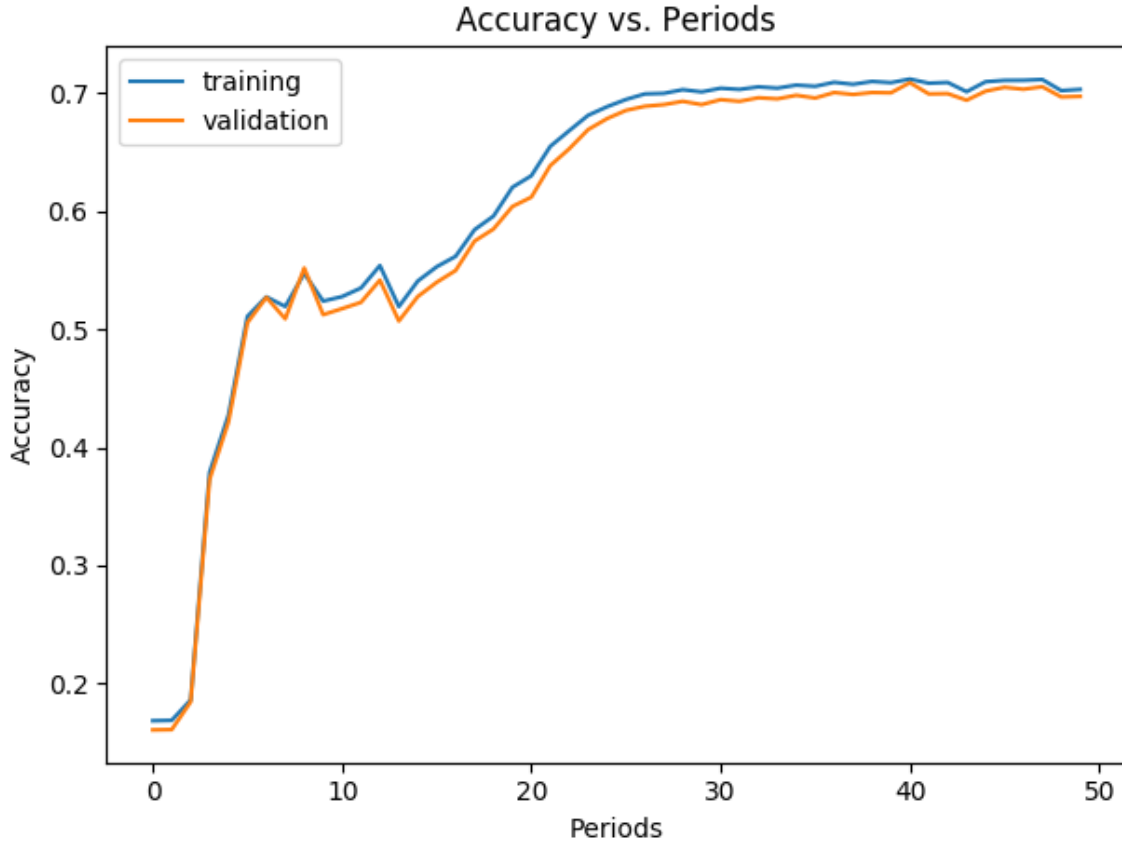


Figure 5.1: Classifier accuracy over periods

Our initial results were off to a good start. However, we wanted to see if we can increase the accuracy of our model.

Our first idea was to do clean up of the data as there can be outliers in the dataset which reduces the accuracy of the classifier. An outlier is a data point in a data set which significantly varies from the rest of the data points in the data set. An outlier can be of two types, global outliers and contextual outliers. A global outlier is the one which significantly varies from the rest of the data set. For example if we are modelling natural temperature on earth, a reading of 500 degrees celsius would be an outlier. A contextual outlier varies from the rest of the data set of a specific class. For example, in our above example a

temperature reading of -40 degrees celsius is an outlier for, lets say, Africa but not for Antarctica.

As we were working with contextualized models, we decided to look for contextual outliers. We first get the dataset of each landmark and plotted the occurrence of authenticated delays. One such graph of a landmark AFNORTHSAFH00 can be seen in figure 5.2 below:

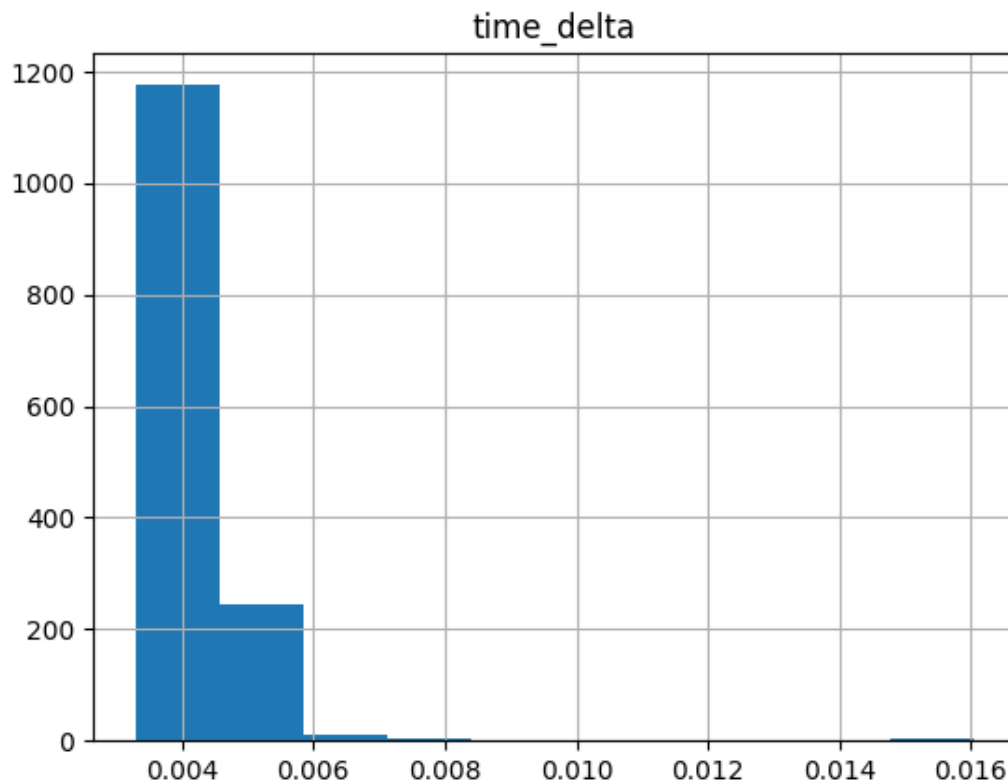


Figure 5.2: Frequency graph of authenticated delay

Another such graph of different landmark can be seen in the figure below

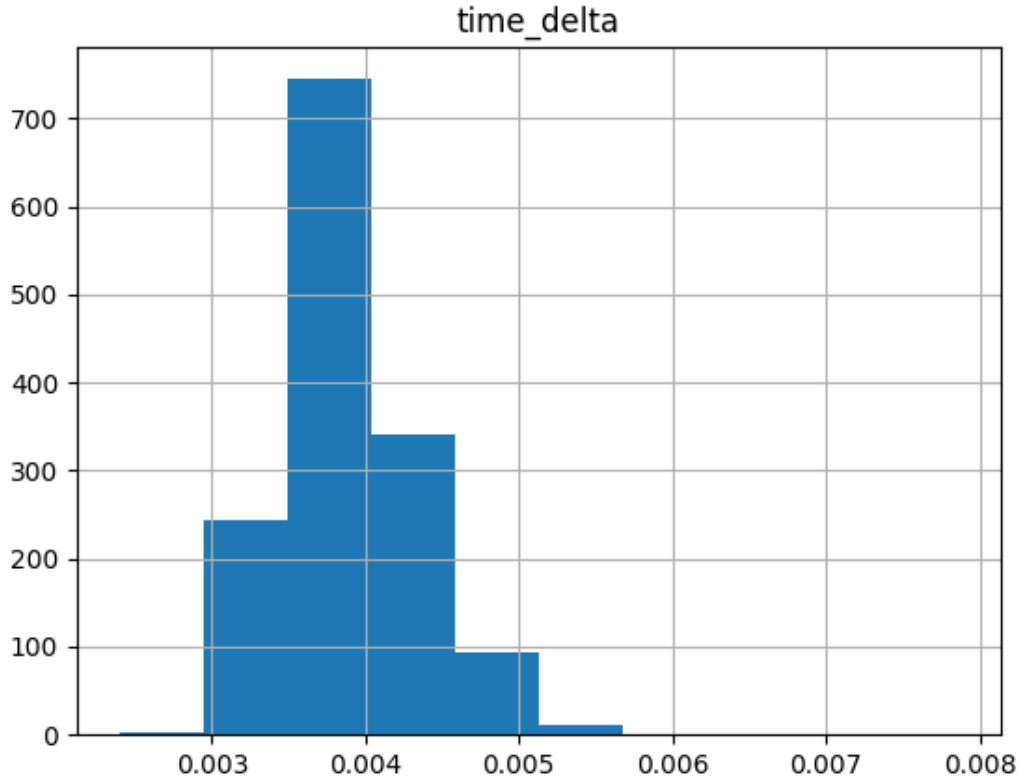


Figure 5.3: Frequency graph of authenticated delay

We can see in the above graphs that most data points are lied in a short range of time but we have some outliers due to which we have long logarithmic tail and we have lognormal distribution. To clip the outliers we could employ different techniques. One such could be filtering where we define a threshold and any value greater than threshold is simply clipped. However the question here would arise that how do we calculate this threshold. We decided to use percentiles. A percentile is a value in a dataset which tells how many other values in the data set, in percentage, are below that value. For example, a percentile of 90 would produce a value in the dataset such that only 10% of the values in the dataset are above the output value. As our data points are quite densely packed so we decided to clip only top and bottom 2 percentile in the dataset. We then again plotted the occurrences of time from cleaned dataset. The corresponding frequency graphs of the above shown

landmarks can be seen below:

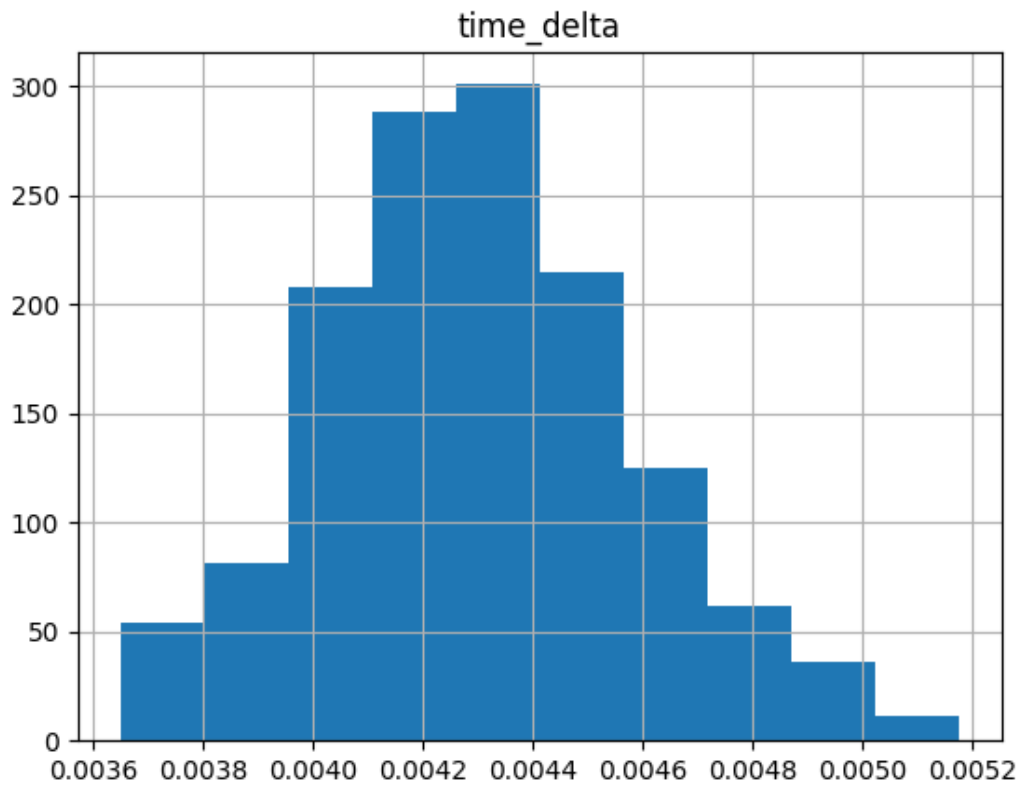


Figure 5.4: Post-filtering frequency graph of authenticated delay

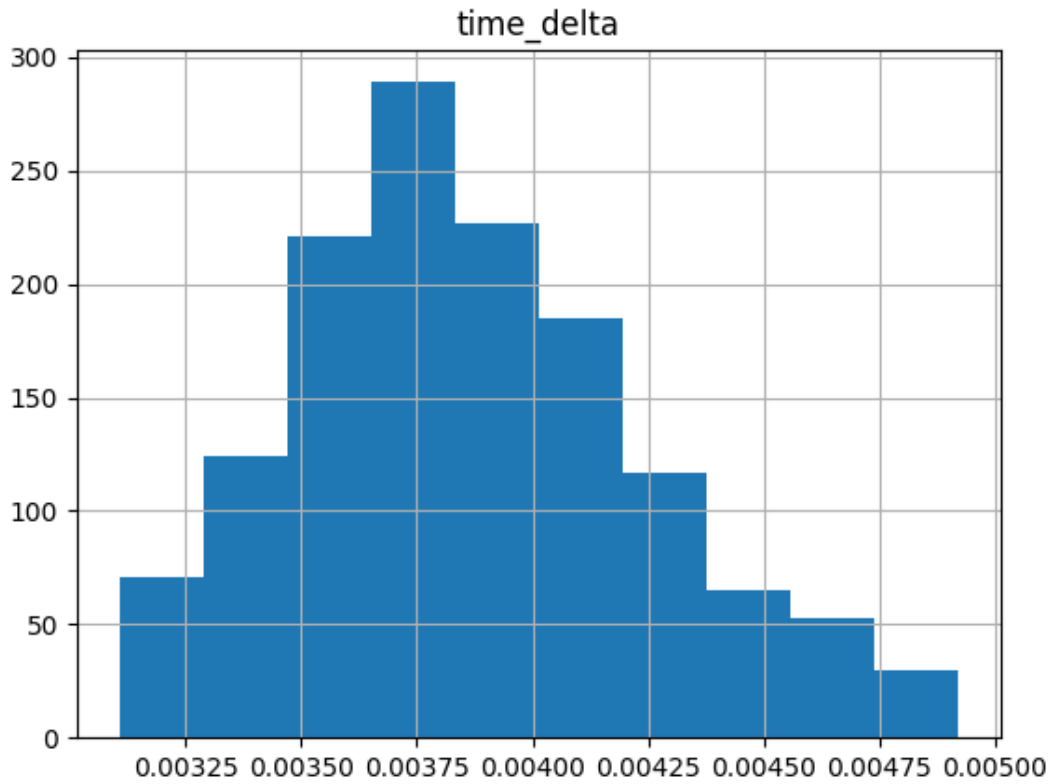


Figure 5.5: Post-filtering frequency graph of authenticated delay

Now we can see that after clipping of the outliers, we have symmetric binomial distribution. This clean dataset, 96% of the original dataset (top and bottom 2% are clipped), is now used to train the classifier. The accuracy over training period graph of this dataset is shown below.

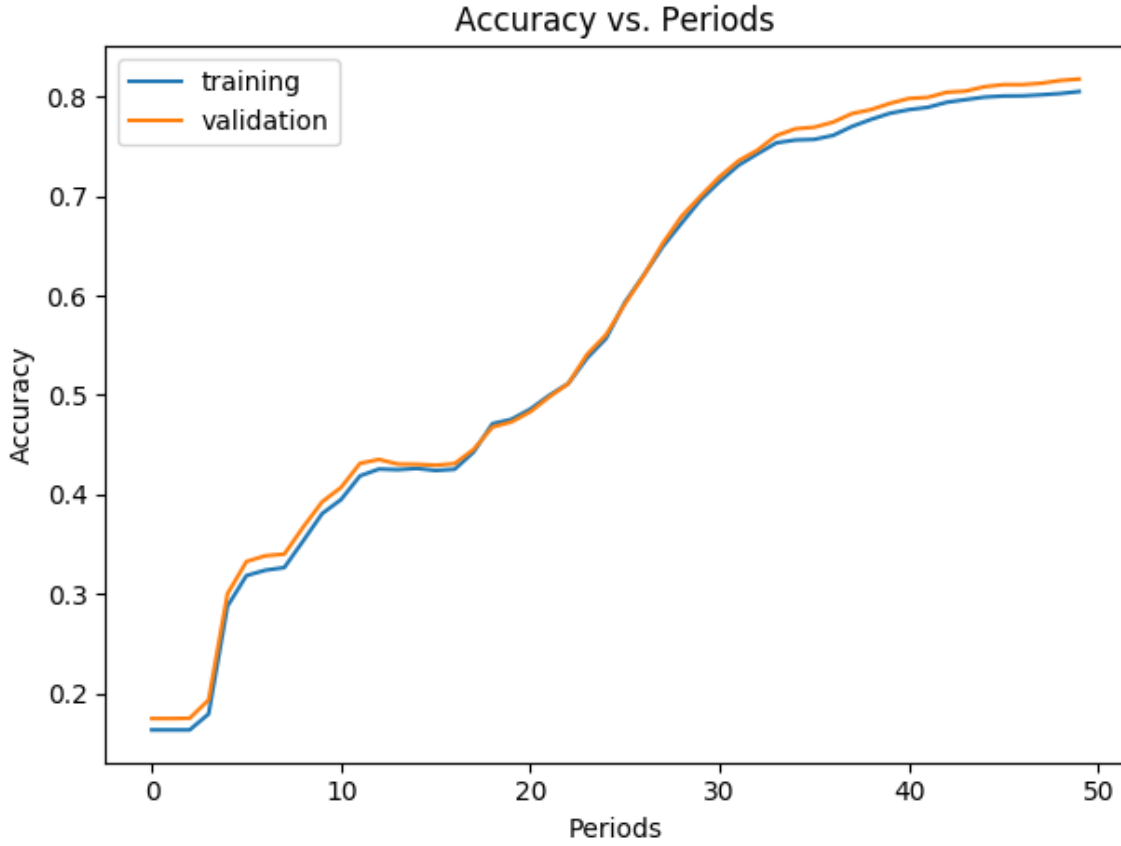


Figure 5.6: Post-filter training accuracy over periods

Here we can see that the final accuracy of the model has reached 82%, an improvement but not significant.

The next idea to increase the accuracy of the model was to include day of the week and time of day in feature vector. The reason for adding these feature vector is that time of day and day of week might play a role in delay measurements like delay can be higher during office hours of the day as compared to none office hours or may have higher delay on work days as compared to weekend due to network load. But to make sure of this we had to do some data analysis. For this we get the delay measurements for each landmark for a week and then separated measurements on the basis of a day. Then we averaged the measurements on hourly basis and plot the average delay over time. Here we need to make sure that time should be globally resolvable. What it means is that for a machine in

Johannesburg, for example, 6 AM would be different than 6 AM on a machine in Victoria and if we are not careful about it we can corrupt the data. However as we used UTC time zone for all our time stamping this issue was resolved as 6 AM UTC will be different at each location but all of them will resolve to one global time instance which we can translate to 6 AM UTC.

The time analysis for the Ireland machine can be seen below

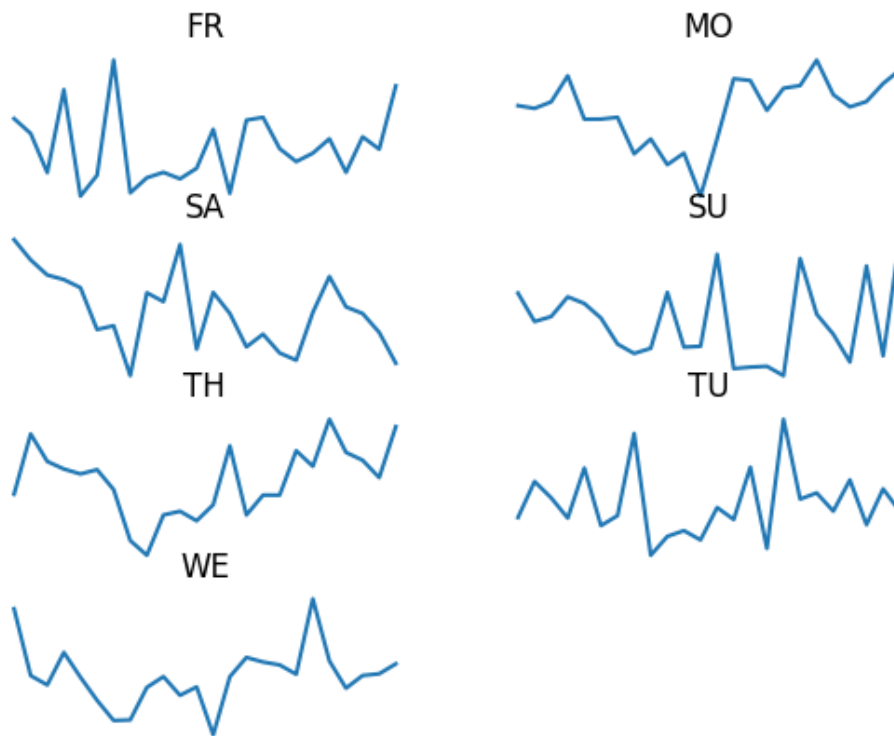


Figure 5.7: Average delay over a week

Here we can see that the general trend for a day is that it starts a bit higher and then it gradually decreases and then gradually increases. In the figure below we can see another example of time analysis of machine in Pune:

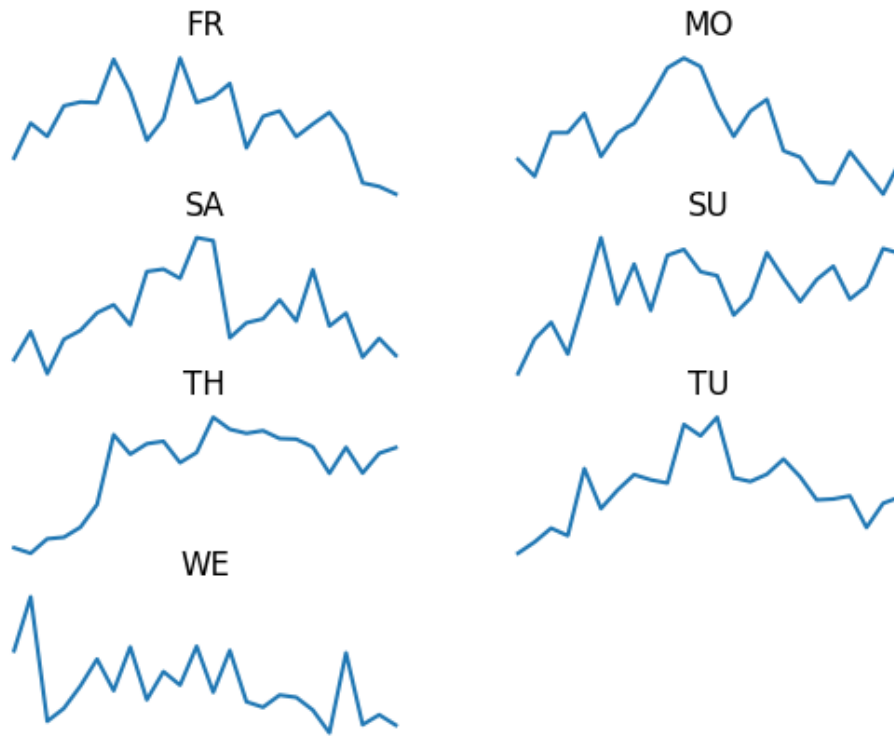


Figure 5.8: Average delay over a week

Here we can see that the general trend is a bit arc like, it starts a bit low and then gradually increases towards the middle of the day and then gradually decreases. Having confirmed our hypothesis we then moved towards creating feature vector for our training. Here we added day of week and time of day as well so our feature vector becomes

Table 5.2: Feature vectors

td	m_hour	m_day	label
T_{al}	H_1	D_1	A
T_{bl}	H_1	D_1	B
T_{cl}	H_1	D_1	C
T_{dl}	H_1	D_1	D
$\vdots$	$\vdots$	$\vdots$	$\vdots$
T_{al}	H_2	D_2	A
$\vdots$	$\vdots$	$\vdots$	$\vdots$

Here T_{al} is the authenticated delay from landmark A at H_1 hour of D_1 day of the week and A is the label or output. We then used this feature vector to train our model but to our surprise the accuracy of the model remained unchanged and still stayed in 80% range. We tried multiple iterations of training with different hyper parameters but the accuracy of the model did not go past 82%

We then decided to do further data analysis and we were soon able to find the reason behind 80% accuracy. Below are few data points for the landmark in Ireland.

Table 5.3: Delay measurement example

td	label
0.4921	00
0.7511	01
0.5013	02
0.3272	03
0.2340	03

We had 6 landmarks and each landmark model have to predict 5 locations. Now if we look at the above data we can see that location 01, 03 and 04 are different quantization

level so the model learns their range to predict the output but if we look at the value of location 00 and 02 we see that they are almost the same hence the model cannot accurately distinguish between these two locations. Now we can see the reason behind the accuracy of 80% as three of the locations (60% of the dataset) were able to be predicted with high probability and remaining two locations (40% of the dataset) were able to be predicted with 50% probability. In other terms we can say that the model was unable to distinguish between two separate locations with the same delay profile.

Now we needed some way to add another constraint to predict the location. So we decided to add all the authenticated delay at time instant t to predict the location of target landmark as opposed to originating landmark. Now we have 6 delays to predict the location of a landmark. Now that even if two landmarks are timewise equi-distant, we can use other delays to “triangulate” the location. It also removes the need to have a different model at each location rather than one model keep tracks of all the contextualization. The model learns the ratio of different delays to predict the location. For 6 landmarks as in our case the feature vector now turns out to be:

Table 5.4: Feature vectors

T0	T1	T2	T3	T4	T5	label
T_{0t1}	T_{1t1}	T_{2t1}	T_{3t1}	T_{4t1}	T_{5t1}	0
T_{0t1}	T_{1t1}	T_{2t1}	T_{3t1}	T_{4t1}	T_{5t1}	1
T_{0t1}	T_{1t1}	T_{2t1}	T_{3t1}	T_{4t1}	T_{5t1}	2
T_{0t1}	T_{1t1}	T_{2t1}	T_{3t1}	T_{4t1}	T_{5t1}	3
T_{0t1}	T_{1t1}	T_{2t1}	T_{3t1}	T_{4t1}	T_{5t1}	4
T_{0t1}	T_{1t1}	T_{2t1}	T_{3t1}	T_{4t1}	T_{5t1}	5
T_{0t2}	T_{1t2}	T_{2t2}	T_{3t2}	T_{4t2}	T_{5t2}	0
T_{0t2}	T_{1t2}	T_{2t2}	T_{3t2}	T_{4t2}	T_{5t2}	1
T_{0t2}	T_{1t2}	T_{2t2}	T_{3t2}	T_{4t2}	T_{5t2}	2
T_{0t2}	T_{1t2}	T_{2t2}	T_{3t2}	T_{4t2}	T_{5t2}	3
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Here T_{nt1} represent the time it takes to establish an SSH connection from landmark n to landmark l at time instant $t1$ with label l . Here we need to make an observation that if the label is 0 at $t1$ then T_{0t1} would be undefined because we cannot make ssh connection to localhost and we are already on that machine. Same is the case with label 1, i.e. T_{1t1} would be undefined. Hence the feature vector would have undefined value where index is equal to label. This type of feature vector is called multi-hot encoding. We can define any value to represent the absence of a value, like 0. A more expressive form of feature vector can then be seen below.

Table 5.5: Feature vectors

T0	T1	T2	T3	T4	T5	label
undefined	T_{1t1}	T_{2t1}	T_{3t1}	T_{4t1}	T_{5t1}	0
T_{0t1}	undefined	T_{2t1}	T_{3t1}	T_{4t1}	T_{5t1}	1
T_{0t1}	T_{1t1}	undefined	T_{3t1}	T_{4t1}	T_{5t1}	2
T_{0t1}	T_{1t1}	T_{2t1}	undefined	T_{4t1}	T_{5t1}	3
T_{0t1}	T_{1t1}	T_{2t1}	T_{3t1}	undefined	T_{5t1}	4
T_{0t1}	T_{1t1}	T_{2t1}	T_{3t1}	T_{4t1}	undefined	5
undefined	T_{1t2}	T_{2t2}	T_{3t2}	T_{4t2}	T_{5t2}	0
T_{0t2}	undefined	T_{2t2}	T_{3t2}	T_{4t2}	T_{5t2}	1
T_{0t2}	T_{1t2}	undefined	T_{3t2}	T_{4t2}	T_{5t2}	2
T_{0t2}	T_{1t2}	T_{2t2}	undefined	T_{4t2}	T_{5t2}	3
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

This feature vector has an inherent problem. The model trained on this feature vector will learn to output the index of undefined value as its output. This model will fail to predict the value in the second attack model when cloud service provider reroute the connection as the model will not care about the difference in delay rather just output undefined value which will not only decrease the accuracy rather make the whole model useless.

To counter this issue we decided to decrease the dimensionality of the feature vector by

1, i.e. in the case of 6 landmarks the feature vector dimensionality is 5 and we omit the value which is undefined. The feature vectors now looks like:

Table 5.6: Feature vectors

T0	T1	T2	T3	T4	label
T_{1t1}	T_{2t1}	T_{3t1}	T_{4t1}	T_{5t1}	0
T_{0t1}	T_{2t1}	T_{3t1}	T_{4t1}	T_{5t1}	1
T_{0t1}	T_{1t1}	T_{3t1}	T_{4t1}	T_{5t1}	2
T_{0t1}	T_{1t1}	T_{2t1}	T_{4t1}	T_{5t1}	3
T_{0t1}	T_{1t1}	T_{2t1}	T_{3t1}	T_{5t1}	4
T_{0t1}	T_{1t1}	T_{2t1}	T_{3t1}	T_{4t1}	5
T_{1t2}	T_{2t2}	T_{3t2}	T_{4t2}	T_{5t2}	0
T_{0t2}	T_{2t2}	T_{3t2}	T_{4t2}	T_{5t2}	1
T_{0t2}	T_{1t2}	T_{3t2}	T_{4t2}	T_{5t2}	2
T_{0t2}	T_{1t2}	T_{2t2}	T_{4t2}	T_{5t2}	3
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Here an important observation is required that we just omitted the undefined values and left shift the remaining values, hence the order of the time deltas remains same for every label for all time instances. This way model learns not only the ratio of different delays but also the context of each label for example (as a reference to example data points given in table before) if T0 is 0.4921, T1 is 0.7511, T2 is 0.5013, T3 is 0.3272, T4 is 0.2340 then label 5. Here another thing is also to be noted that we are just writing time deltas here but as time delta also depends on the time of day and day of the week as explained before we also need those data points in our feature vector. Hence complete feature vector now looks like

Table 5.7: Feature vectors

T0	T1	T2	T3	T4	m_hour	m_day	label
T_{1t1}	T_{2t1}	T_{3t1}	T_{4t1}	T_{5t1}	H_1	D_1	0
T_{0t1}	T_{2t1}	T_{3t1}	T_{4t1}	T_{5t1}	H_1	D_1	1
T_{0t1}	T_{1t1}	T_{3t1}	T_{4t1}	T_{5t1}	H_1	D_1	2
T_{0t1}	T_{1t1}	T_{2t1}	T_{4t1}	T_{5t1}	H_1	D_1	3
T_{0t1}	T_{1t1}	T_{2t1}	T_{3t1}	T_{5t1}	H_1	D_1	4
T_{0t1}	T_{1t1}	T_{2t1}	T_{3t1}	T_{4t1}	H_1	D_1	5
T_{1t2}	T_{2t2}	T_{3t2}	T_{4t2}	T_{5t2}	H_1	D_1	0
T_{0t2}	T_{2t2}	T_{3t2}	T_{4t2}	T_{5t2}	H_1	D_1	1
T_{0t2}	T_{1t2}	T_{3t2}	T_{4t2}	T_{5t2}	H_1	D_1	2
T_{0t2}	T_{1t2}	T_{2t2}	T_{4t2}	T_{5t2}	H_1	D_1	3
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

We used these feature vectors to train our model. To our delight the accuracy of the model jumped from 80% to 97%. The graph of accuracy over training periods can be seen in the figure below.

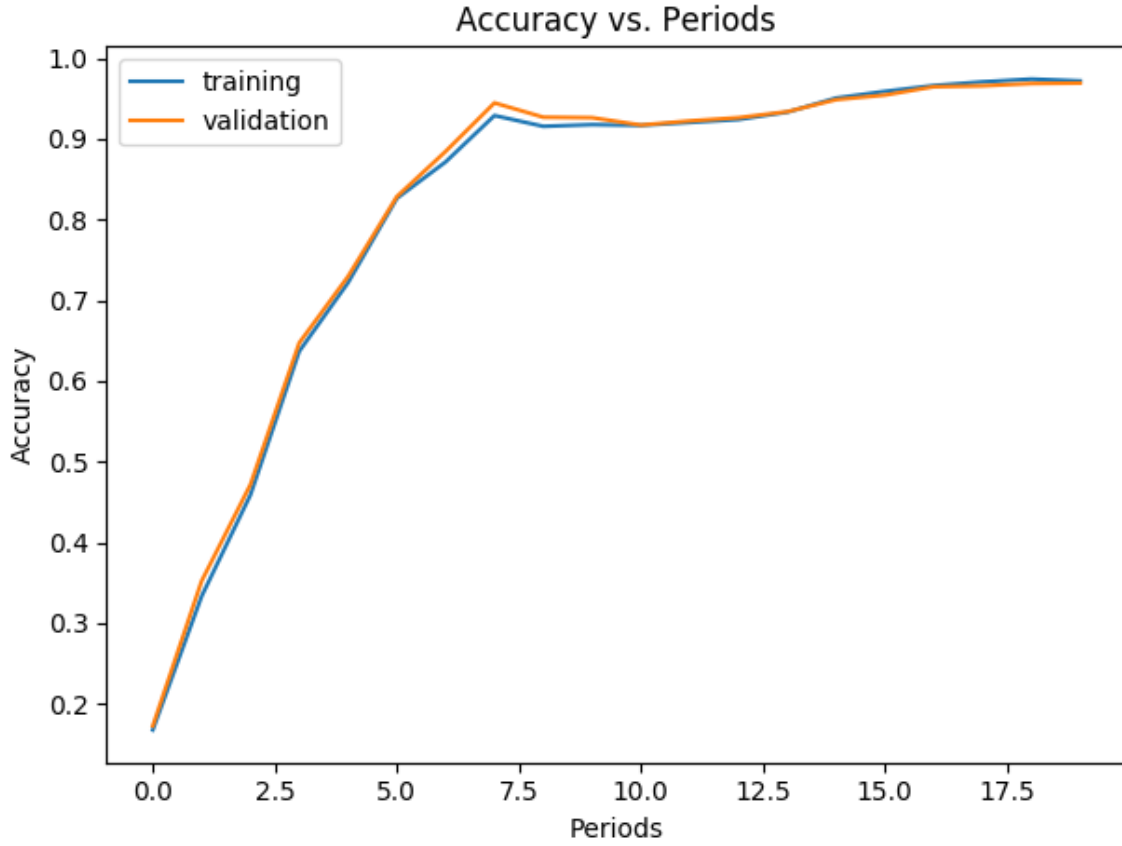


Figure 5.9: Final training accuracy over periods

97% accuracy implies that the location of a cloud resource can be validated with a high probability. We can validate the location of a cloud resource with 0.03 error probability with one data point. This probability can be further reduced by running multiple iterations of independent tests. For example 2 independently taken delay measurements can validate the location of a cloud resource, each with 0.03 error probability. However, as per general rule of probability multiplication the error probability of the whole experiment is reduced to 0.0009 ($0.03 * 0.03$)

To further visualize the accuracy and precision of the model, we generated a confusion matrix which can be seen in the figure below

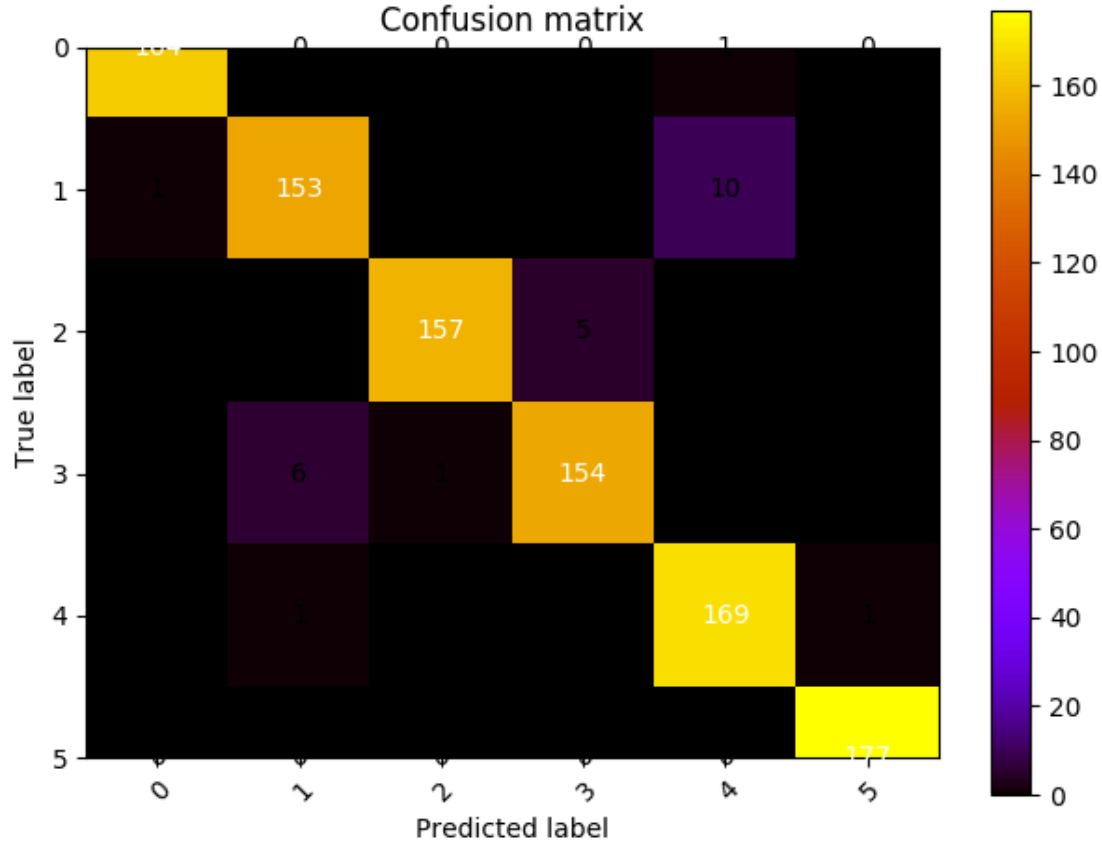


Figure 5.10: Confusion matrix

We can see in the figure that our model can validate the location a cloud resource with high recall and high precision. We can see from the graph that highest precision (a value of 1) was obtained by EUWESTEUFH05 and APSEASTASFH02 obtained the lowest precision with the value of 0.93. In terms of recall EUWESTEUFH05 is the top scorer again with 0.99 and lowest recall was achieved by USEASTUS1FH04 with 0.93. Hence our lower limit of both recall and precision is 0.93.

Our next task was to validate the location of cloud resource in second attack model, that is when malicious service provider installs a gateway at claimed location but routes the traffic to a different location. To validate the location of a cloud resource in this attack model, we predicted the locations of our machines using routed delay dataset by the same classification model which was trained on training dataset in previous step. I will

summarize our delay dataset here again. We created a tunnel from USEASTUS1FH04 (label 4) to AFNORTHSAFH00 (label 0) such that all traffic aiming for USEASTUS1FH04 is routed towards AFNORTHSAFH00. To better visualize the results of this classification model, we again build the confusion matrix of relayed dataset predictions which can be seen in figure 5.11 below

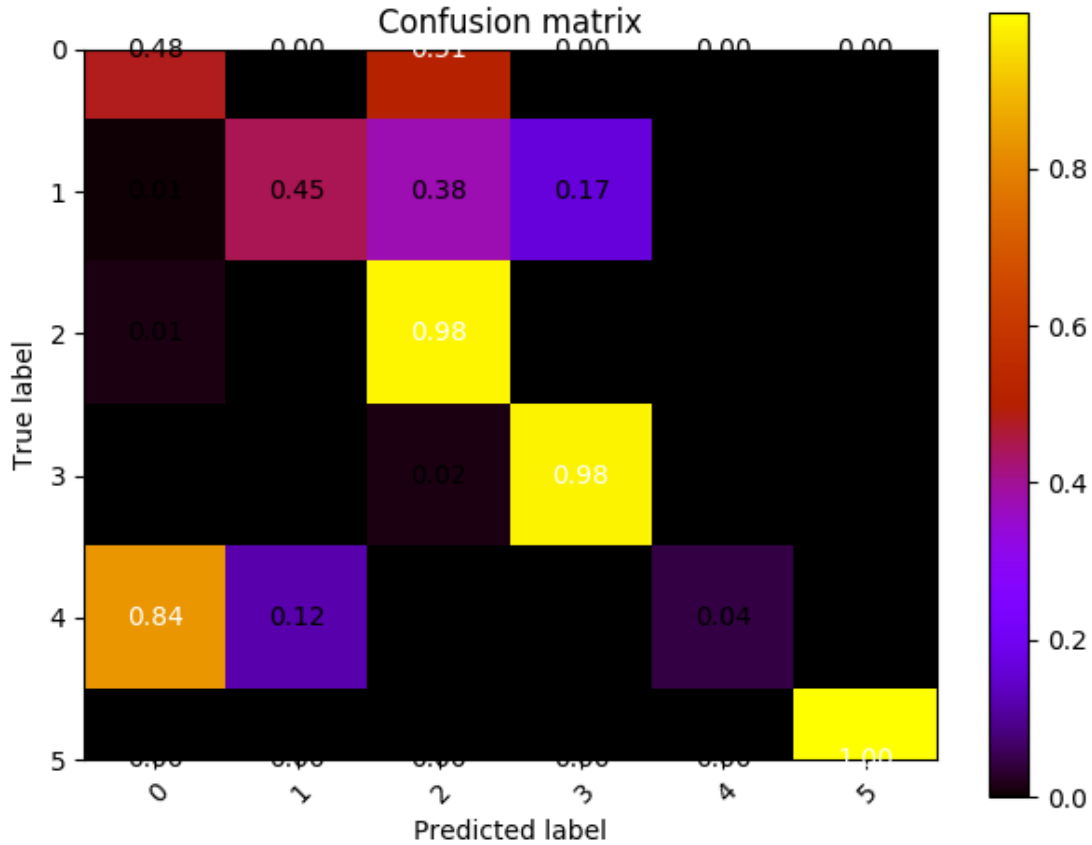


Figure 5.11: Confusion matrix of relayed dataset

Lets analyze the precision of each class one by one. The precision of class 5 remained very high i.e. 1. Class 4 (routed location) showed interesting results. It was able successfully invalidate the claimed location as a very minute quantity of the result predicted that location. The interesting result is that it also predicted location 0 (the actual location of resource) on 84% of dataset. Hence not only it invalidated its false location but it also also predicted the correct location with high enough probability. The precision of class 2 and 3

remained also very high with 0.99 and 0.98 respectively. However, the precision of class 0 and 1 dropped drastically to less than 50%. We believe one reason of reduced accuracy could be small dataset as we collected routed data for a couple of hours only. The lower the dataset size, higher it has the chances of having momentary fluctuations. Another reason could be actual network topology change as the training data collection and routed data collection were 5 months apart. However further tests are required to validate this theory.

6 Conclusion

In this chapter we will wrap up our thesis. First we will compare our idea with other similar approaches. In the next section we will explain the shortcomings of our system and based on these shortcomings we will propose some future recommendations to make our system better.

As we have seen in the previous chapter, our location validation technique through classification produced noteworthy numbers in recall and precision, we can now compare it with other related approaches. Most of the literature that we have studied in this field and explained in chapter 2 covers the estimation of geolocation. However as we are working with validation rather than estimation, we can not directly compare the results with those as they are solving the regression problem and we are working with classification, two disparate problems. However Stephanow et al [7] also proposed a validation technique so we can compare our results with their. In their paper they claimed to have achieved 92.7% accuracy overall. As we are using more modern machine learning techniques and more extensive features to predict the location, we were able to reach the accuracy of 97% with precision and recall reaching minimum to 0.93.

6.1 Shortcomings

We would like to address a few shortcomings of our approach in this section. Our system relies on the complete picture of the whole network, it relies on measurements from all the landmarks. Unless delay measurement for all the landmarks at time instant T are not captured, it will not be used as a training data in training the classifier. This will make the system a bit stringent. Even one failed or crashed measuring probe will render the whole

system useless. Another shortcoming of our system is that it assumes that the network is correct during the training data collection. As it relies on cloud infrastructure during the training data collection, it can not detect an already relocated networks however it will be able to detect further changes in the network topology. At the global level, network topology is a dynamic thing and it can change quite frequently, which may impact on delay measurements. Our system does not take care of these continuous changes in the network and it may increase the misclassification rate of the model.

6.2 Future Recommendations

As proposed by Stephanow et al, we could employ a continuous training technique to continuously train our model with some thresholding. As one regular changes in the network does not impact the delay measurement by too much as compared to getting the resources from different regions, we can define a threshold of model change. This way we can simultaneously continuously update the classification model and detect the the unwanted changes in the network as well.

Bibliography

- [1] Amazon. *Amazon Web Services (AWS) - Cloud Computing Services*. <https://aws.amazon.com/>. [Online; accessed May 28, 2019]. 2019.
- [2] Microsoft. *Microsoft Azure Cloud Computing Platform & Services*. <https://azure.microsoft.com/>. [Online; accessed May 28, 2019]. 2019.
- [3] Google. *Cloud Computing Services*. <https://cloud.google.com/>. [Online; accessed May 28, 2019]. 2019.
- [4] E. Directive. “95/46/EC of the European Parliament and of the Council of 24 October 1995 on the protection of individuals with regard to the processing of personal data and on the free movement of such data”. In: *Official Journal of the EC* 23.6 (1995).
- [5] OAIC. *Australian Privacy Principles quick reference*. <https://www.oaic.gov.au/privacy/australian-privacy-principles/australian-privacy-principles-quick-reference/>. [Online; accessed May 28, 2019]. 2014.
- [6] B. Gueye, A. Ziviani, M. Crovella, and S. Fdida. “Constraint-based geolocation of internet hosts”. In: *IEEE/ACM Transactions on Networking (TON)* 14.6 (2006), pp. 1219–1232.
- [7] P. Stephanow and K. Khajehmoogahi. “Towards continuous security certification of Software-as-a-Service applications using web application testing techniques”. In: *2017 IEEE 31st International Conference on Advanced Information Networking and Applications (AINA)*. IEEE. 2017, pp. 931–938.
- [8] M. Eskandari, A. S. De Oliveira, and B. Crispo. “Vloc: An approach to verify the physical location of a virtual machine in cloud”. In: *2014 IEEE 6th International Conference on Cloud Computing Technology and Science*. IEEE. 2014, pp. 86–94.

- [9] E. Katz-Bassett, J. P. John, A. Krishnamurthy, D. Wetherall, T. Anderson, and Y. Chawathe. "Towards IP geolocation using delay and topology measurements". In: *Proceedings of the 6th ACM SIGCOMM conference on Internet measurement*. ACM. 2006, pp. 71–84.
- [10] V. N. Padmanabhan and L. Subramanian. "An investigation of geographic mapping techniques for Internet hosts". In: *ACM SIGCOMM Computer Communication Review*. Vol. 31. 4. ACM. 2001, pp. 173–185.
- [11] M. Gondree and Z. N. Peterson. "Geolocation of data in the cloud". In: *Proceedings of the third ACM conference on Data and application security and privacy*. ACM. 2013, pp. 25–36.
- [12] T. Ylonen and C. Lonvick. "RFC 4253: The secure shell (SSH) transport layer protocol". In: *The Internet Society* (2006).
- [13] J. Tahir. *The leading native Python SSHv2 protocol library*. <https://github.com/jawadtahir/paramiko>. 2019.
- [14] C. Davis, I. Dickinson, T. Goodwin, and P. Vixie. "A means for expressing location information in the domain name system". In: (1996).
- [15] Moore, D., Periakaruppan, R., Donohoe, J., & Claffy, K. *Where in the world is net-geo.caida.org*. http://www.caida.org/publications/papers/2000/inet_netgeo/inet_netgeo.html. [Online; accessed May 28, 2019]. 2000.
- [16] G. S. Malkin. "Traceroute using an IP option". In: (1993).
- [17] P. Bahl, V. N. Padmanabhan, V. Bahl, and V. Padmanabhan. "RADAR: An in-building RF-based user location and tracking system". In: (2000).
- [18] C. Bovy, H. Mertodimedjo, G. Hooghiemstra, H. Uijterwaal, and P. Van Mieghem. "Analysis of end-to-end delay measurements in Internet". In: *Proc. of the Passive and Active Measurement Workshop-PAM*. Vol. 2002. sn. 2002.
- [19] M. J. Arif, S. Karunasekera, and S. Kulkarni. "GeoWeight: internet host geolocation based on a probability model for latency measurements". In: *Proceedings of the*

- Thirty-Third Australasian Conferenc on Computer Science-Volume 102*. Australian Computer Society, Inc. 2010, pp. 89–98.
- [20] *IP Address API - Accurate IP Address Location API*. <http://www.ipaddressapi.com/>. [Online; accessed May 28, 2019]. 2014.
- [21] P. Stephanow, M. Moein, and C. Banse. “Continuous location validation of cloud service components”. In: *2017 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. IEEE. 2017, pp. 255–262.
- [22] Amazon. *Why Cloud Infrastructure Matters*. Amazon Global Infrastructure. <https://aws.amazon.com/about-aws/global-infrastructure/>. [Online; accessed Nov 28, 2019]. 2019.
- [23] T. Ylonen and C. Lonvick. “RFC 4251: The secure shell (SSH) protocol architecture”. In: *Internet Engineering Task Force* (2006).
- [24] S. Lehtinen and C. Lonvick. “The Secure Shell (SSH) Protocol Assigned Numbers”. In: (2006).
- [25] T. Ylonen and C. Lonvick. “The secure shell (SSH) authentication protocol”. In: (2006).
- [26] T. Ylonen and C. Lonvick. “RFC 4254 The Secure Shell (SSH) Connection Protocol”. In: *Internet Engineering Task Force (IETF): Network Working Group* (2006).
- [27] J. Schlyter and W. Griffin. “Using DNS to securely publish secure shell (SSH) key fingerprints”. In: (2006).
- [28] M. Stuij. *AWS explained: Global Infrastructure*. <https://www.inqdo.com/wp-content/uploads/2018/03/global-infrastructure.png>. [Online; accessed Dec 04, 2019]. 2019.
- [29] J. Forcier. *Paramiko*. <http://www.paramiko.org/>. [Online; accessed Dec 04, 2019]. 2019.
- [30] *The leading native Python SSHv2 protocol libraryy*. <https://github.com/paramiko/paramiko>. 2019.

- [31] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, et al. "Tensorflow: A system for large-scale machine learning". In: *12th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 16)*. 2016, pp. 265–283.
- [32] F. Chollet. *Keras*. <https://keras.io>. [Online; accessed Dec 04, 2019]. 2015.
- [33] Docker. *What Is a Container?* <https://www.docker.com/resources/what-container>. [Online; accessed Dec 04, 2019]. 2019.